

SQL Cheat Sheet: Accessing Databases using Python

Topic	Syntax	Description	Example
connect()	<code>sqlited.connect()</code>	Create a new database and open a database connection to allow <code>sqlited</code> to work with it. Call <code>sqlited.connect()</code> to create a connection to the database <code>INSTRUCTOR.db</code> in the current working directory, implicitly creating it if it does not exist.	<pre>import sqlited con = sqlited.connect("INSTRUCTOR.db")</pre>
cursor()	<code>con.cursor()</code>	To execute SQL statements and fetch results from SQL queries, use a database cursor. Call <code>con.cursor()</code> to create the Cursor.	<pre>cursor_obj = con.cursor()</pre>
execute()	<code>cursor_obj.execute()</code>	The <code>execute</code> method in Python's <code>SQLite</code> library allows to perform SQL commands, including retrieving data from a table using a query like "Select from table name". When you execute this command, the result is obtained as a collection of table data stored in an object, typically in the form of a list of lists.	<pre>cursor_obj.execute("""insert into INSTRUCTOR values (1, 'Sam', 'Mugla', 'TURKISH', 'CA')""")</pre>
fetchall()	<code>cursor_obj.fetchall()</code>	The <code>fetchall()</code> method in Python retrieves all the rows from the result set of a query and presents them as a list of tuples.	<pre>statement = """SELECT * FROM INSTRUCTOR""" cursor_obj.execute(statement) result_obj = cursor_obj.fetchall() for row_obj in result_obj: print(row_obj)</pre>
fetchmany()	<code>cursor_obj.fetchmany()</code>	The <code>fetchmany()</code> method retrieves the subsequent group of rows from the result set of a query rather than just a single row. To fetch a few rows from the table, use <code>fetchmany(numberofrows)</code> and mention how many rows you want to fetch.	<pre>statement = """SELECT * FROM INSTRUCTOR""" cursor_obj.execute(statement) result_obj = cursor_obj.fetchmany(3) for row_obj in result_obj: print(row_obj)</pre>
read_sql_query()	<code>read_sql_query()</code>	<code>read_sql_query()</code> is a function provided by the <code>Pandas</code> library in Python, and it is not specific to <code>MySQL</code> . It is a generic function used for executing SQL queries on various database systems, including <code>MySQL</code> , and retrieving the results as a <code>Pandas DataFrame</code> .	<pre>df = pd.read_sql_query("select * from instructor()", con)</pre>
shape	<code>dataFrame.shape</code>	It provides a tuple indicating the shape of a <code>DataFrame</code> or <code>Series</code> , represented as (number of rows, number of columns).	<pre>df.shape</pre>
close()	<code>con.close()</code>	<code>con.close()</code> is a method used to close the connection to a <code>MySQL</code> database. When called, it terminates the connection, releasing any associated resources and ensuring the connection is no longer active. This is important for managing database connections efficiently and preventing resource leaks in your <code>MySQL</code> database interactions.	<pre>con.close()</pre>
CREATE TABLE	<code>CREATE TABLE table_name (column datatype constraints, column2 datatype constraints, ...);</code>	The <code>create</code> <code>table</code> statement is used to define and create a new table within a database. It specifies the table's name, the structure of its columns (including data types and constraints), and any additional properties such as indexes. This statement essentially sets up the blueprint for organizing and storing data in a structured format within the database.	<pre>CREATE TABLE INSTRUCTOR2 (id INT(4) NOT NULL, first_name VARCHAR(30), last_name VARCHAR(30), test_score INT);</pre>
barplot()	<code>seaborn.barplot(x="test_variable", y="test_variable", data=data)</code>	<code>seaborn.barplot()</code> is a function in the <code>Seaborn</code> Python data visualization library used to create a bar plot, also known as a bar chart. It is particularly used to display the relationship between a categorical variable and a numeric variable by showing the average value for each category.	<pre>import seaborn seaborn.barplot(x="test_score", y="frequency", data=dataFrame)</pre>
read_csv()	<code>df = pd.read_csv("file_path.csv")</code>	<code>read_csv()</code> is a function in Python's <code>Pandas</code> library used for reading data from a Comma-Separated Values (CSV) file and loading it into a <code>Pandas DataFrame</code> . It's a common method for working with tabular data stored in CSV format.	<pre>import pandas df = pandas.read_csv("https://data.cityofchicago.org/resource/jsoq-8kaf.csv")</pre>
to_sql()	<code>df.to_sql("table_name", index=False)</code>	<code>df.to_sql()</code> is a method in <code>Pandas</code> , a Python data manipulation library used to write the contents of a <code>DataFrame</code> to a <code>SQL</code> database. It allows to take data from a <code>DataFrame</code> and store it structurally within a <code>SQL</code> database table.	<pre>import pandas df = pandas.read_csv("https://data.cityofchicago.org/resource/jsoq-8kaf.csv") df.to_sql("Chicago, socioeconomic_data", con, if_exists="replace", index=False, method="multi")</pre>
read_sql()	<code>df = pd.read_sql(read_query, con)</code>	<code>read_sql()</code> is a function provided by the <code>Pandas</code> library in Python for executing <code>SQL</code> queries and retrieving the results into a <code>DataFrame</code> from an <code>SQL</code> database. It's a convenient way to integrate <code>SQL</code> database interactions into your data analysis workflows.	<pre>sqlQuery = "select * from INSTRUCTOR" df = pandas.read_sql(sqlQuery, con)</pre>

Db2

Topic	Syntax	Description	Example
connect()	<code>con = ibm_db.connect("DRIVER=IBM_DB;DSN=ibmcom/PROD;PORT=5000;USER=IBMUSER;PASSWORD='';", "", "")</code>	<code>ibm_db.connect()</code> is a Python function provided by the <code>ibm_db</code> library which is used for establishing a connection to an <code>IBM Db2</code> or <code>IBM Db2 Warehouse</code> database. It's commonly used in applications that need to interact with <code>IBM Db2</code> databases from Python.	<pre>import ibm_db con = ibm_db.connect("DRIVER=IBM_DB;DSN=ibmcom/PROD;PORT=5000;USER=IBMUSER;PASSWORD='';", "", "")</pre>
server_info()	<code>ibm_db.server_info()</code>	<code>ibm_db.server_info()</code> is a Python function provided by the <code>ibm_db</code> library, which is used to retrieve information about the <code>IBM Db2</code> server to which you are connected.	<pre>server = ibm_db.server_info(con) print ("DBNAME: ", server.DB_NAME) print ("DBSCHEMA: ", server.DB_SCHEMA) print ("DB_NAME: ", server.DB_NAME)</pre>
close()	<code>con.close()</code>	<code>con.close()</code> is a method used to close the connection to a <code>db2</code> database. When called, it terminates the connection, releasing any associated resources and ensuring the connection is no longer active. This is important for managing database connections efficiently and preventing resource leaks in your <code>db2</code> database interactions.	<pre>con.close()</pre>
exec_immediate()	<code>sql_statement = "SQL statement goes here" ret = ibm_db.exec_immediate(con, sql_statement)</code>	<code>con.exec_immediate()</code> is a Python function provided by the <code>ibm_db</code> library, which is used to execute an <code>SQL</code> statement immediately without the need to prepare or bind it. It's commonly used for executing <code>SQL</code> statements that don't require input parameters or don't need to be prepared in advance.	<pre># Lets first drop the table INSTRUCTOR in case it exists from a previous attempt. dropQuery = "drop table INSTRUCTOR" dropRet = ibm_db.exec_immediate(con, dropQuery)</pre>

Author(s)

Abhishek Garguja
D.M. Naidu

